

Finding descendants of elements in JavaScript

Learn the theory at the following links:

[firstElementChild](#) [lastElementChild](#) [children](#)

firstElementChild property

The `firstElementChild` property contains the first child element. All tags that are directly located inside a block are considered child elements. If the element has no children, then `null` is returned.

Syntax

```
1 element.firstElementChild;
```

Example

Let's get a content of the element's first child:

```
1 <div id="parent">
2   <p>1</p>
3   <p>2</p>
4 </div>

1 let parent = document.querySelector('#parent');
2 let text = parent.firstElementChild.textContent;
3
4 console.log(text);
```

The code execution result:

```
1 '1'
```

Example

And now an element has no children and therefore `null` will be displayed:

```
1 <div id="parent"></div>

1 let parent = document.querySelector('#parent');
2 console.log(parent.firstElementChild);
```

The code execution result:

```
1 null
```

lastElementChild property

The `lastElementChild` property stores the last child element. All tags that are directly located inside a block are considered child elements. If the element has no children, then `null` is returned.

Syntax

```
1 | element.lastElementChild;
```

Example

We get a contents of the last child of an element:

```
1 | <div id="parent">
2 |   <p>1</p>
3 |   <p>2</p>
4 | </div>

1 | let parent = document.querySelector('#parent');
2 | let text = parent.lastElementChild.textContent;
3 |
4 | console.log(text);
```

The code execution result:

```
1 | '2'
```

Example

And now an element has no children and therefore `null` will be displayed:

```
1 | <div id="parent"></div>

1 | let parent = document.querySelector('#parent');
2 | console.log(parent.lastElementChild);
```

The code execution result:

```
1 | null
```

children property

The `children` property stores a `pseudo-array` of child elements. All tags that are directly located inside a block are considered child elements.

Syntax

```
1 | element.children;
```

Example

Let's loop through all the element's children and display their content:

```

1 <div id="parent">
2   <p>1</p>
3   <p>2</p>
4   <p>3</p>
5   <p>4</p>
6   <p>5</p>
7 </div>

```

```

1 let parent = document.querySelector('#parent');
2 let elems = parent.children;
3
4 for (let elem of elems) {
5   console.log(elem.textContent);
6 }

```

№1

©jsPmSrEl

Given the element **#elem**:

```

1 <ul id="elem">
2   <li>text</li>
3   <li>text</li>
4   <li>text</li>
5   <li>text</li>
6   <li>text</li>
7 </ul>

```

Find the first child of this element and make its text red.

№2

©jsPmSrEl

Given the element **#elem**:

```

1 <ul id="elem">
2   <li>text</li>
3   <li>text</li>
4   <li>text</li>
5   <li>text</li>
6   <li>text</li>
7 </ul>

```

Find the last child of this element and make its text red.

№3

©jsPmSrEl

Given the element **#elem**:

```

1 <ul id="elem">
2   <li>text</li>
3   <li>text</li>
4   <li>text</li>
5   <li>text</li>
6   <li>text</li>

```

```
7 | </ul>
```

Find all the children of this element and append the text '!' to the end of them.

Finding parents of elements in JavaScript

Learn the theory at the following links:

[parentElement](#)

parentElement property

The `parentElement` property contains a parent element.

Syntax

```
1 | element.parentElement;
```

Example

Let's obtain the parent of the `#elem` element and display its `id`:

```
1 | <div id="parent">
2 |   <p id="elem"></p>
3 | </div>

1 | let elem = document.querySelector('#elem');
2 | let id = elem.parentElement.id;
3 |
4 | console.log(id);
```

The code execution result:

```
1 | 'parent'
```

№1

JS PMS REI

Given the element `#elem`:

```
1 | <div>
2 |   <ul>
3 |     <li>text</li>
4 |     <li>text</li>
5 |     <li id="elem">text</li>
6 |     <li>text</li>
7 |     <li>text</li>
8 |   </ul>
9 | </div>
```

Search for all parents of elements in JavaScript

Learn the theory at the following links:

[closest](#)

closest method

The `closest` method searches for the closest parent element that matches the specified CSS selector, while the element itself is also included in the search.

Syntax

```
1 element.closest('selector');
```

Example

Let's find among the parents of an element the element with the class `www`:

```
1 <div class="www" id="parent2">
2   <div class="ggg" id="parent1">
3     <p class="zzz" id="child"></p>
4   </div>
5 </div>
```

```
1 let elem = document.querySelector('#child');
2 let parent = elem.closest('.www');
3
4 console.log(parent.id);
```

The code execution result:

```
1 'parent2'
```

Example

Let's find among the parents of an element the element with the class `www`. As a result, we will get a reference to the element itself, since it has this class itself:

```
1 <div class="www" id="parent2">
2   <div class="www" id="parent1">
3     <p class="www" id="child"></p>
4   </div>
5 </div>
```

```
1 let elem = document.querySelector('#child');
2 let parent = elem.closest('.www');
3 console.log(parent.id);
```

The code execution result:

```
1 'child'
```

No 1

⊗jsPmSrAI

Given an element:

```
1 <header>
2   <div>
3     <p>
4       <span id="elem"></span>
5     </p>
6   </div>
7 </header>
```

Find the closest parent of this element, which is the `div` tag.

No 2

⊗jsPmS

Given an element:

```
1 <header>
2   <div class="www">
3     <p class="www">
4       <span id="elem"></span>
5     </p>
6   </div>
7 </header>
```

Find the closest parent of this element, which is an element with the class `www`.

Search for siblings of elements in JavaScript

Learn the theory at the following links:

[previousElementSibling](#) [nextElementSibling](#)

previousElementSibling property

The `previousElementSibling` property contains the previous element that is in the same parent. If there is no such element, then `null` is returned.

Syntax

```
1 element.previousElementSibling;
```

Example

The element `#elem` is given. Let's display the text of his neighbor from above:

```
1 <p>sibling</p>
2 <p id="elem">elem</p>

1 let elem = document.querySelector('#elem');
2 let text = elem.previousElementSibling.textContent;
```

```
3  
4 console.log(text);
```

The code execution result:

```
1 'sibling'
```

Example

If there is no neighbor above or it is located not in the parent of our element, `null` is also returned:

```
1 <p>sibling</p>  
2 <div>  
3   <p id="elem">elem</p>  
4 </div>
```

```
1 let elem = document.querySelector('#elem');  
2 console.log(elem.previousElementSibling);
```

The code execution result:

```
1 null
```

nextElementSibling property

The `nextElementSibling` property contains the next element in the same parent. If there is no such element, `null` is returned.

Syntax

```
1 element.nextElementSibling;
```

Example

The element `#elem` is given. Let's display the text of his neighbor from below:

```
1 <p id="elem">elem</p>  
2 <p>sibling</p>
```

```
1 let elem = document.querySelector('#elem');  
2 let text = elem.nextElementSibling.textContent;  
3  
4 console.log(text);
```

The code execution result:

```
1 'sibling'
```

Example

If there is no bottom neighbor or it is located not in the parent of our element, `null` is also returned:

```
1 <div>
2   <p id="elem">elem</p>
3 </div>
4 <p>sibling</p>

1 let elem = document.querySelector('#elem');
2 console.log(elem.nextElementSibling);
```

The code execution result:

```
1 null
```

№ 1

©jsPmSr

Given the element `#elem`:

```
1 <ul>
2   <li>text</li>
3   <li>text</li>
4   <li id="elem">text</li>
5   <li>text</li>
6   <li>text</li>
7 </ul>
```

Find its upper sibling element and add the text '!' to the end of it.

№ 2

©jsPmSrS

Given the element `#elem`:

```
1 <ul>
2   <li>text</li>
3   <li>text</li>
4   <li id="elem">text</li>
5   <li>text</li>
6   <li>text</li>
7 </ul>
```

Find its bottom sibling element and add the text '!' to the end of it.

№ 3

©jsPmSrS1

Given the element `#elem`:

```
1 <ul>
2   <li>text</li>
```



```

3 | <li>text</li>
4 | <li id="elem">text</li>
5 | <li>text</li>
6 | <li>text</li>
7 | </ul>

```

Find its bottom sibling of its bottom sibling (the next element after the next one) and add the text '!' to the end of it.

No4

✉jsPmSrs

Given the element **#elem**:

```

1 | <ul>
2 |   <li>text1</li>
3 |   <li>text2</li>
4 |   <li id="elem">text3</li>
5 |   <li>text4</li>
6 |   <li>text5</li>
7 | </ul>

```

Swap the text of its upper sibling and the text of its bottom sibling.

Getting of element by id in JavaScript

Learn the theory at the following links:

[getElementById](#)

getElementById method

The `getElementById` method allows you to get a page element by its `id` attribute. With the resulting element, it will be possible to perform various manipulations: change its text, attributes, CSS styles, and so on.

Syntax

```
1 | document.getElementById(element id);
```

Example

Let's get an `input` by its `id` and change the content of the `value` attribute:

```

1 | <input id="elem" value="???">

1 | let elem = document.getElementById('elem');
2 | elem.value = '!!!';

```

The code execution result:

```
1 | <input id="elem" value="!!!">
```

Given the element `#elem`:

```
1 <div id="elem"></div>
```

Get this element using the `getElementById` method and set some text to it.

Getting of element by tag name in JavaScript

Learn the theory at the following links:

[getElementsByTagName](#)

getElementsByTagName method

The `getElementsByTagName` method allows you to get page elements by a tag name. With the retrieved elements, you can perform various manipulations: change their text, attributes, CSS styles, and so on.

Syntax

```
1 document.getElementsByTagName(tag name);
```

Example

Let's get all the paragraphs and change their text using the property `innerHTML`:

```
1 <p>elem 1</p>
2 <p>elem 2</p>
3 <p>elem 3</p>

1 let elems = document.getElementsByTagName('p');
2
3 for (let elem of elems) {
4   elem.innerHTML = '!!!';
5 }
```

The code execution result:

```
1 <p>!!!</p>
2 <p>!!!</p>
3 <p>!!!</p>
```

Given a list:

```
1 <ul>
2   <li>text</li>
3   <li>text</li>
```

```
4 | <li>text</li>
5 | <li>text</li>
6 | <li>text</li>
7 | </ul>
```

Get all the `li` tags by tag name and make their text red.

Getting of element by class name in JavaScript

Learn the theory at the following links:

[getElementsByClassName](#)

getElementsByClassName method

The `getElementsByClassName` method allows you to get page elements by their class specified in the `class` attribute. It will be possible to perform various manipulations with the retrieved elements: change their text, attributes, CSS styles, and so on.

Syntax

```
1 | document.getElementsByClassName(class name);
```

Example

Let's get all elements with the class `www` and change their text using the `innerHTML` property:

```
1 | <p class="www">elem 1</p>
2 | <p class="www">elem 2</p>
3 | <p class="www">elem 3</p>

1 | let elems = document.getElementsByClassName('www');
2 |
3 | for (let elem of elems) {
4 |   elem.innerHTML = '!!!';
5 | }
```

The code execution result:

```
1 | <p class="www">!!!</p>
2 | <p class="www">!!!</p>
3 | <p class="www">!!!</p>
```

№ 1

www.jsfiddle.com

Given the elements:

```
1 | <p class="www">elem 1</p>
2 | <p class="www">elem 2</p>
3 | <p class="www">elem 3</p>
```

Get these elements by class name and make their text red.

Search for elements inside another element in JavaScript

Suppose we have some parent element, and other elements inside it:

```
1 <div id="parent">
2   <div class="child">text</div>
3   <div class="child">text</div>
4   <div class="child">text</div>
5 </div>
```

Let the reference to the parent has already been derived into a variable:

```
1 let parent = document.querySelector('#parent');
```

In this case, if necessary, we can search for the selector within that parent, rather than the entire document. To do this, the search method must be applied not to `document`, but to a variable containing our parent.

For example, let's find elements with the `child` class inside our parent:

```
1 let elems = parent.querySelectorAll('.child');
```



№ 1

© jsPmSrIE

Given the element `#parent`:

```
1 <div id="parent">
2   <p class="www">text</p>
3   <p class="www">text</p>
4   <p class="www">text</p>
5   <p class="ggg">text</p>
6   <p class="ggg">text</p>
7   <p class="ggg">text</p>
8 </div>
```

```
1 let parent = document.querySelector('#parent');
```

Find elements with the class `www` inside the parent and store them in the variable `elems1`. Then find elements with the class `ggg` inside the parent and store them in the variable `elems2`.